

MASLD Phenotyping Using Unsupervised Machine Learning

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':  
##  
##   filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(cluster)  
library(ggplot2)  
library(reshape2)  
library(NbClust)  
library(e1071)
```

```
##  
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:ggplot2':  
##  
##   element
```

```
library(fmsb)  
library(mclust)
```

```
## Package 'mclust' version 6.1.3  
## Type 'citation("mclust")' for citing this R package in publications.
```

```
##  
## Attaching package: 'mclust'
```

```
## The following object is masked from 'package:dplyr':  
##  
##   count
```

```
library(MASS)
```

```
##  
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':  
##  
##      select
```

```
library(gridExtra)
```

```
##  
## Attaching package: 'gridExtra'
```

```
## The following object is masked from 'package:dplyr':  
##  
##      combine
```

```
library(grid)  
  
while (!is.null(dev.list())) dev.off()  
  
# ---- EDIT THIS ONE LINE ONLY – path to your prepared cohort CSV ----  
INPUT_FILE <- "D:/MASLD/cohort_data.csv"  
K_RANGE <- 3:10  
GAP_B <- 5  
NBOOT <- 10
```

Section 1: Load, Filter, Prepare Clustering Data

```
if (!file.exists(INPUT_FILE)) stop("File not found: ", INPUT_FILE)  
  
masld_cohort <- read.csv(INPUT_FILE, stringsAsFactors = FALSE, check.names = FALSE)  
cat("Rows:", nrow(masld_cohort), "| Columns:", ncol(masld_cohort), "\n")
```

```
## Rows: 11933 | Columns: 24
```

```

masld_clean <- masld_cohort %>%
  filter(is.na(Urine_Pregnancy) | Urine_Pregnancy != 1) %>%
  filter(is.na(HepB_Core_Ab) | HepB_Core_Ab != 1) %>%
  filter(is.na(HepC_Ab) | !(HepC_Ab %in% c(1, 4))) %>%
  filter(
    is.na(Alcohol_Days) | is.na(Gender) |
    (!(Alcohol_Days %in% c(777, 999)) &
    !((Gender == 1 & Alcohol_Days >= 2) |
    (Gender == 2 & Alcohol_Days >= 1)))
  ) %>%
  filter(is.na(Elastography_Status) | !(Elastography_Status %in% c(2, 3, 4))) %>%
  filter(CAP_Median >= 288)

crit1 <- masld_clean$BMI >= 25
crit2 <- masld_clean$Fasting_Glucose >= 100 | masld_clean$HbA1c >= 5.7 | masld_clean$Diabetes
== 1
crit3 <- masld_clean$SBP_AVG >= 130
crit4 <- masld_clean$Triglycerides >= 150
crit5 <- (masld_clean$Gender == 1 & masld_clean$HDL <= 40) | (masld_clean$Gender == 2 & masld
_clean$HDL <= 50)
has_cmrfr <- rowSums(cbind(crit1, crit2, crit3, crit4, crit5), na.rm = TRUE) >= 1

masld_clean <- masld_clean %>% filter((Age >= 18 | is.na(Age)) & has_cmrfr)
cat("Filtered cohort rows:", nrow(masld_clean), "\n")

```

```
## Filtered cohort rows: 709
```

```

clustering_vars <- c("Age", "BMI", "HbA1c", "ALT", "LDL", "Triglycerides",
  "Waist", "SBP_AVG", "Fasting_Glucose")
fasting_vars <- c("LDL", "Triglycerides", "Fasting_Glucose")
impute_vars <- setdiff(clustering_vars, fasting_vars)

df <- masld_clean[complete.cases(masld_clean[, fasting_vars]), c("SEQN", clustering_vars)]
for (v in impute_vars) df[[v]][is.na(df[[v]])] <- median(df[[v]], na.rm = TRUE)
cat("Rows after listwise deletion + imputation:", nrow(df), "\n")

```

```
## Rows after listwise deletion + imputation: 353
```

```

temp_scaled <- scale(df[, clustering_vars])
outlier_rows <- apply(temp_scaled, 1, function(r) any(abs(r) >= 5))
df_raw_before <- df
df <- df[!outlier_rows, ]
cat("Outliers removed:", sum(outlier_rows), "| Final N:", nrow(df), "\n")

```

```
## Outliers removed: 3 | Final N: 350
```

```

scaled <- scale(df[, clustering_vars])
rownames(scaled) <- df$SEQN
dmat <- dist(scaled, method = "euclidean")

```

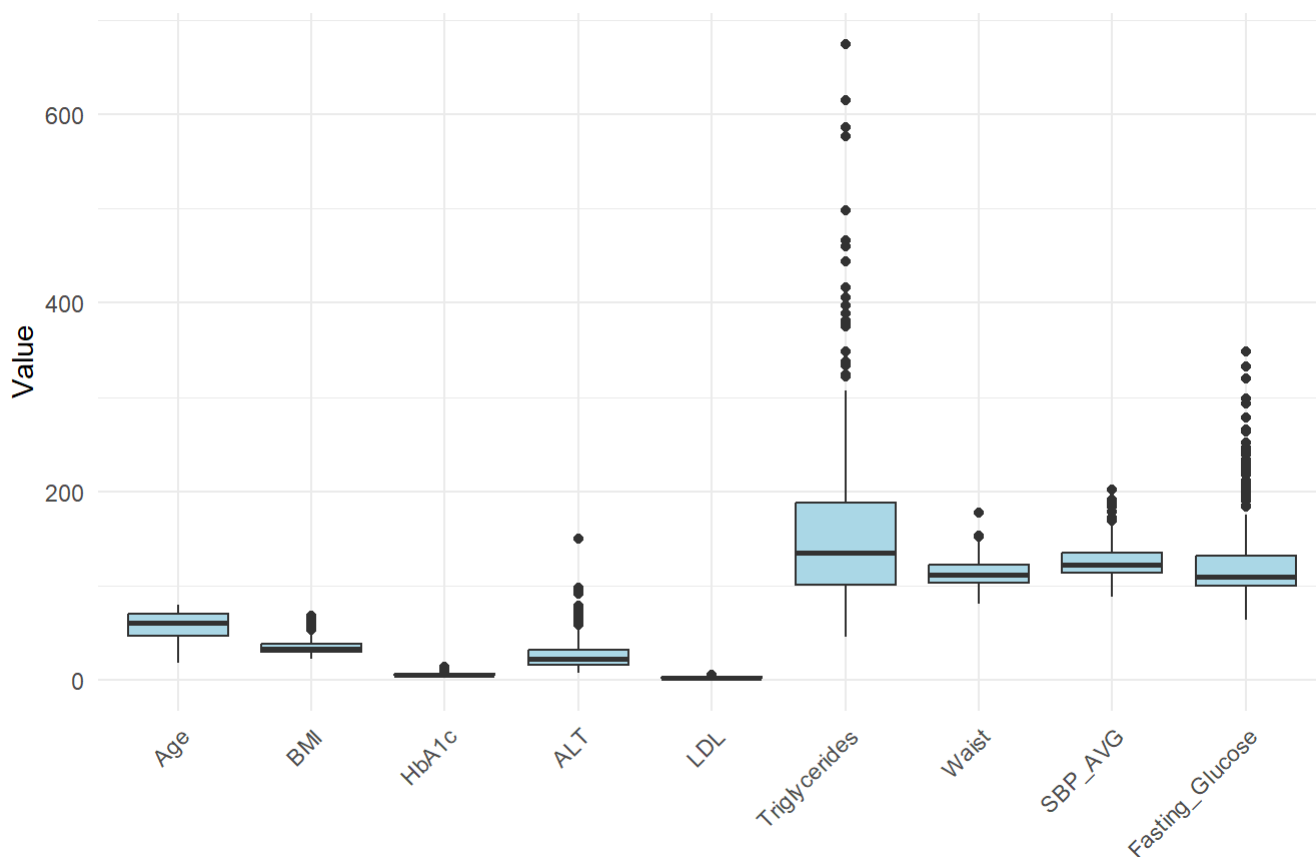
Section 2: Before / After Normalization

```
before_long <- melt(df_raw_before[, clustering_vars], variable.name = "Variable", value.name = "Value")
```

```
## No id variables; using all as measure variables
```

```
ggplot(before_long, aes(Variable, Value)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Before Normalization (raw units, incl. outliers)", x = "", y = "Value") +
  theme_minimal() + theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Before Normalization (raw units, incl. outliers)

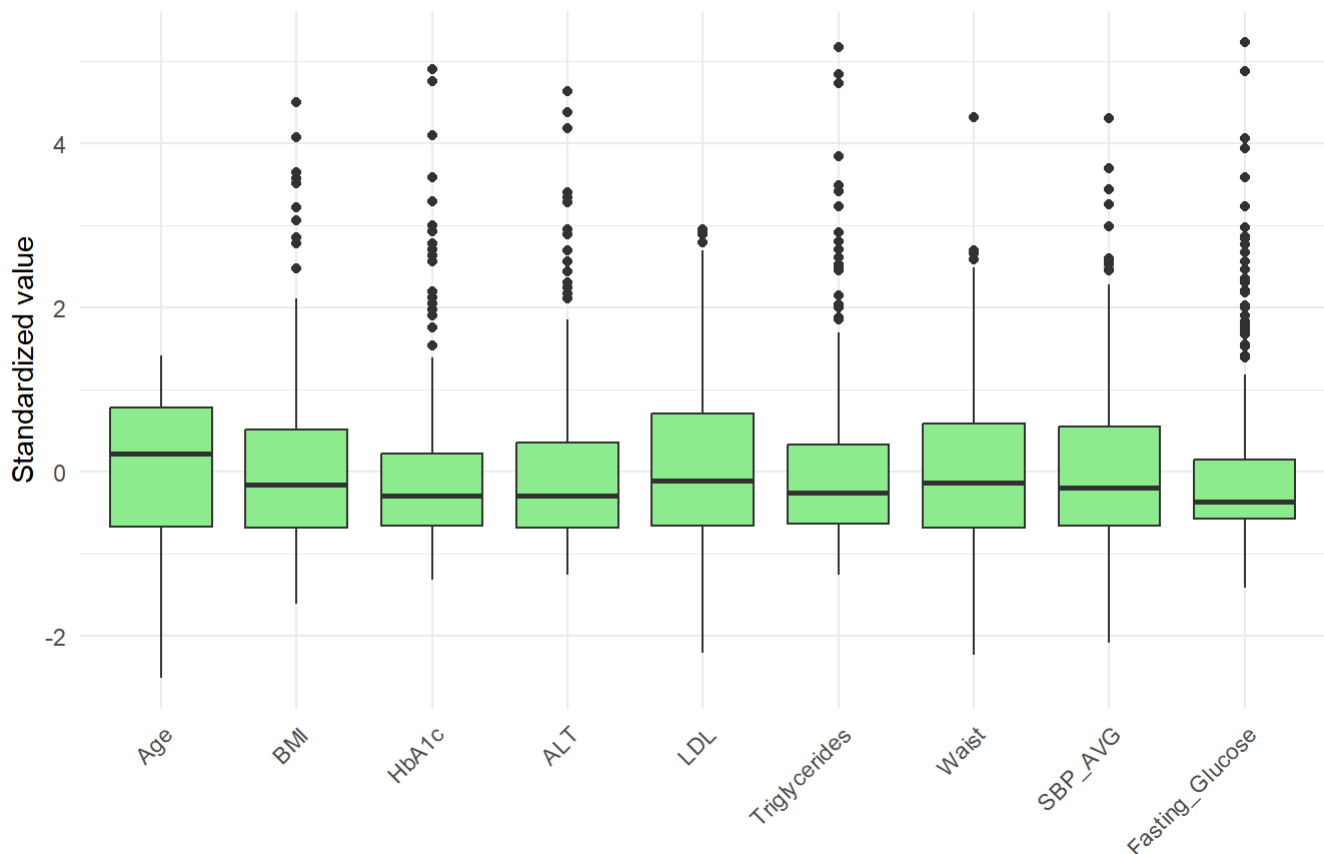


```
after_long <- melt(as.data.frame(scaled), variable.name = "Variable", value.name = "Value")
```

```
## No id variables; using all as measure variables
```

```
ggplot(after_long, aes(Variable, Value)) +
  geom_boxplot(fill = "lightgreen") +
  labs(title = "After Normalization (z-scores, outliers removed)", x = "", y = "Standardized value") +
  theme_minimal() + theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

After Normalization (z-scores, outliers removed)



```
get_gap_best_k <- function(gap_obj) {
  tab <- as.data.frame(gap_obj$Tab)
  list(max_gap_k = which.max(tab$gap),
       best_k_1se = maxSE(tab$gap, tab$SE.sim, method = "firstSEmax"))
}
```

Section 3: PAM — Euclidean

```
pam_euc_sil <- c(); pam_euc_models <- list()
for (k in K_RANGE) {
  set.seed(24)
  m <- pam(scaled, k = k, metric = "euclidean", nstart = NBOOT)
  pam_euc_models[[as.character(k)]] <- m
  pam_euc_sil[as.character(k)] <- mean(silhouette(m)[, 3])
}
pam_euc_table <- data.frame(k = K_RANGE, avg_silhouette = round(pam_euc_sil, 4))
print(pam_euc_table, row.names = FALSE)
```

```
## k avg_silhouette
## 3 0.1568
## 4 0.1225
## 5 0.1232
## 6 0.1323
## 7 0.1156
## 8 0.1129
## 9 0.1227
## 10 0.1201
```

```
pam_euc_best_k <- pam_euc_table$k[which.max(pam_euc_table$avg_silhouette)]
pam_euc_best <- pam_euc_models[[as.character(pam_euc_best_k)]]
cat("Best k (PAM Euclidean):", pam_euc_best_k, "\n")
```

```
## Best k (PAM Euclidean): 3
```

```
set.seed(24)
gap_pam_euc <- clusGap(scaled, FUNcluster = function(x, k) {
  set.seed(24); list(cluster = pam(x, k, metric = "euclidean", nstart = NBOOT)$clustering)
}, K.max = max(K_RANGE), B = GAP_B)
gap_pam_euc_res <- get_gap_best_k(gap_pam_euc)
cat("Gap – max-gap k:", gap_pam_euc_res$max_gap_k, "| 1-SE k:", gap_pam_euc_res$best_k_1se,
    "\n")
```

```
## Gap – max-gap k: 10 | 1-SE k: 1
```

Section 4: PAM — Manhattan (main method)

```
pam_man_sil <- c(); pam_man_models <- list()
for (k in K_RANGE) {
  set.seed(24)
  m <- pam(scaled, k = k, metric = "manhattan", nstart = NBOOT)
  pam_man_models[[as.character(k)]] <- m
  pam_man_sil[as.character(k)] <- mean(silhouette(m)[, 3])
}
pam_man_table <- data.frame(k = K_RANGE, avg_silhouette = round(pam_man_sil, 4))
print(pam_man_table, row.names = FALSE)
```

```
## k avg_silhouette
## 3 0.1544
## 4 0.1531
## 5 0.1173
## 6 0.1233
## 7 0.1221
## 8 0.1288
## 9 0.1037
## 10 0.1031
```

```
pam_man_best_k <- pam_man_table$k[which.max(pam_man_table$avg_silhouette)]
pam_man_best <- pam_man_models[[as.character(pam_man_best_k)]]
cat("Best k (PAM Manhattan):", pam_man_best_k, "\n")
```

```
## Best k (PAM Manhattan): 3
```

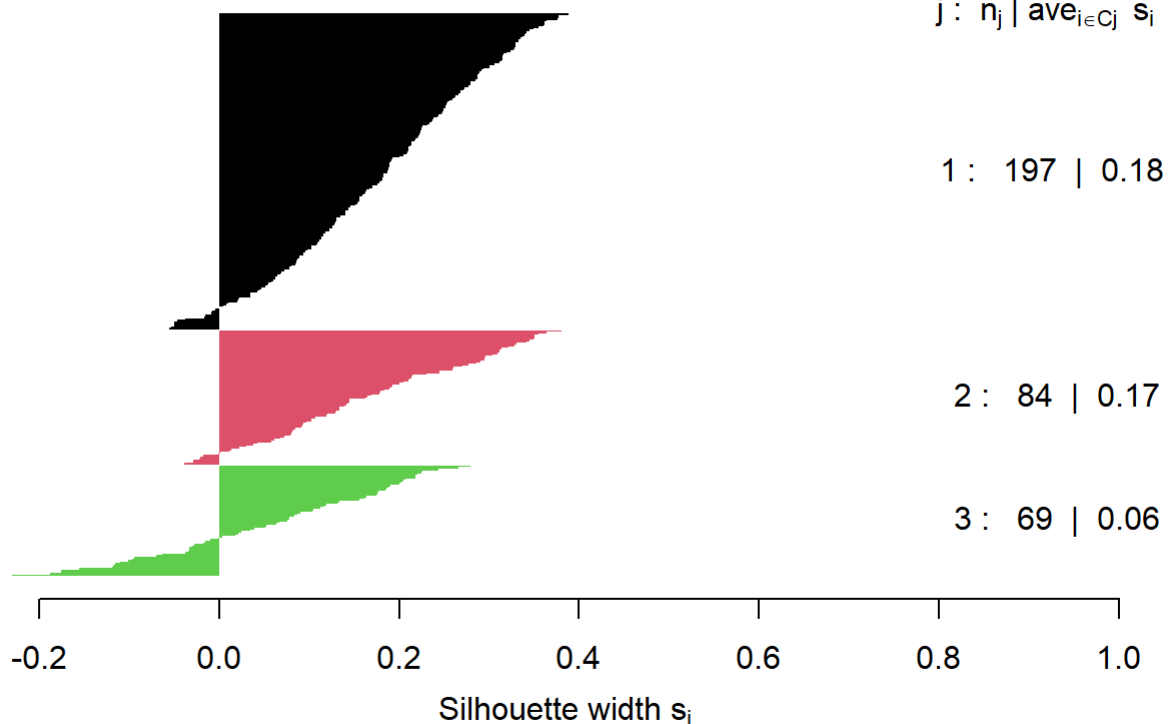
```
set.seed(24)
gap_pam_man <- clusGap(scaled, FUNcluster = function(x, k) {
  set.seed(24); list(cluster = pam(x, k, metric = "manhattan", nstart = NBOOT)$clustering)
}, K.max = max(K_RANGE), B = GAP_B)
gap_pam_man_res <- get_gap_best_k(gap_pam_man)
cat("Gap - max-gap k:", gap_pam_man_res$max_gap_k, "| 1-SE k:", gap_pam_man_res$best_k_1se,
    "\n")
```

```
## Gap - max-gap k: 8 | 1-SE k: 1
```

```
plot(silhouette(pam_man_best), main = paste("PAM Manhattan Silhouette (k =", pam_man_best_k,
    ")"),
     col = 1:pam_man_best_k, border = NA)
```

PAM Manhattan Silhouette (k = 3)

n = 350



Section 5: K-means

```
km_sil <- c(); km_models <- list()
for (k in K_RANGE) {
  set.seed(24)
  m <- kmeans(scaled, centers = k, nstart = NBOOT, iter.max = 1000)
  km_models[[as.character(k)]] <- m
  km_sil[as.character(k)] <- mean(silhouette(m$cluster, dmat)[, 3])
}
km_table <- data.frame(k = K_RANGE, avg_silhouette = round(km_sil, 4))
print(km_table, row.names = FALSE)
```

```
##    k avg_silhouette
##    3          0.1748
##    4          0.1655
##    5          0.1490
##    6          0.1463
##    7          0.1599
##    8          0.1538
##    9          0.1473
##   10          0.1465
```

```
km_best_k <- km_table$k[which.max(km_table$avg_silhouette)]
km_best <- km_models[[as.character(km_best_k)]]
cat("Best k (K-means):", km_best_k, "\n")
```

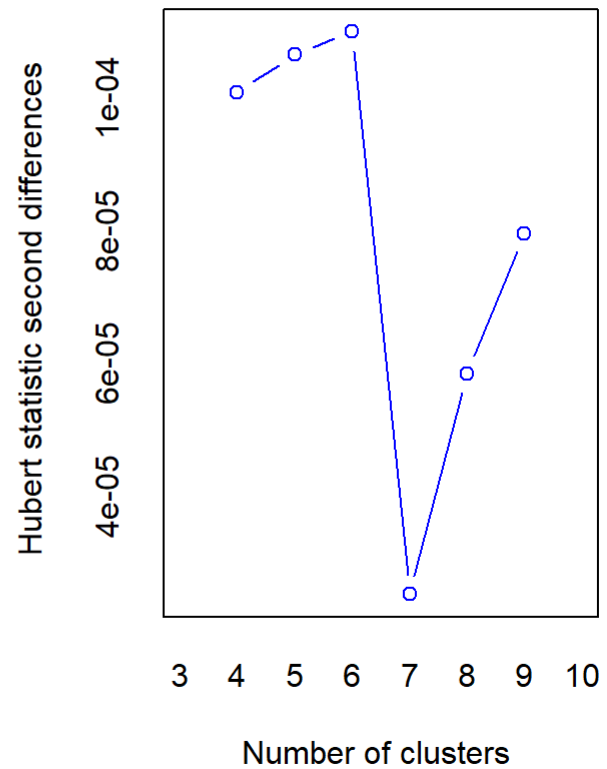
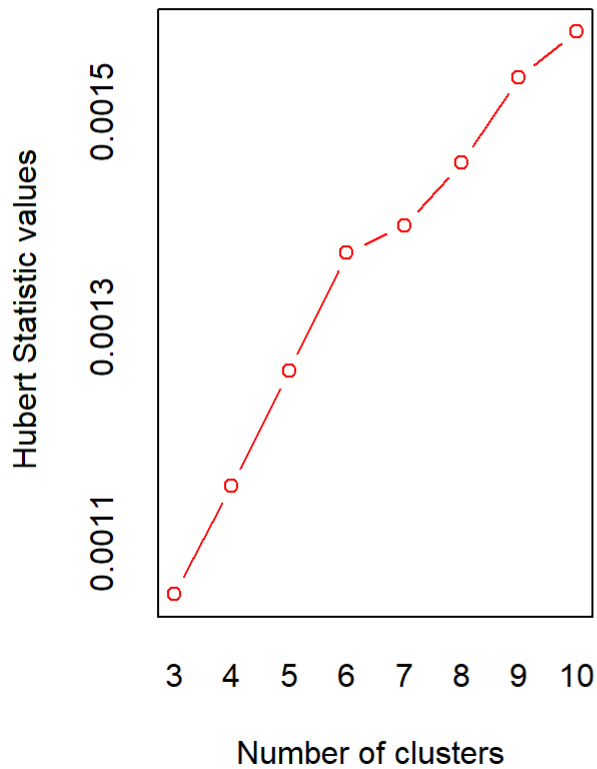
```
## Best k (K-means): 3
```

```
set.seed(24)
gap_km <- clusGap(scaled, FUNcluster = function(x, k) {
  set.seed(24); list(cluster = kmeans(x, centers = k, nstart = NBOOT, iter.max = 1000)$cluster)
}, K.max = max(K_RANGE), B = GAP_B)
gap_km_res <- get_gap_best_k(gap_km)
cat("Gap - max-gap k:", gap_km_res$max_gap_k, "| 1-SE k:", gap_km_res$best_k_1se, "\n")
```

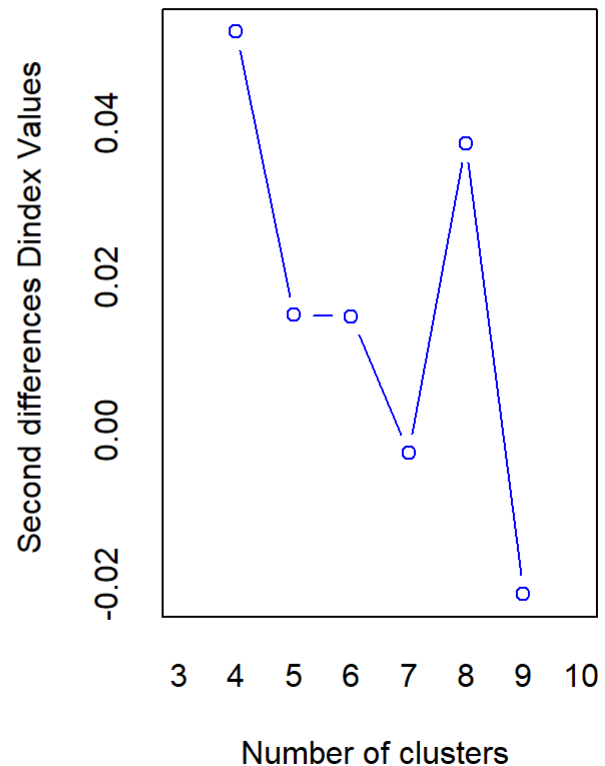
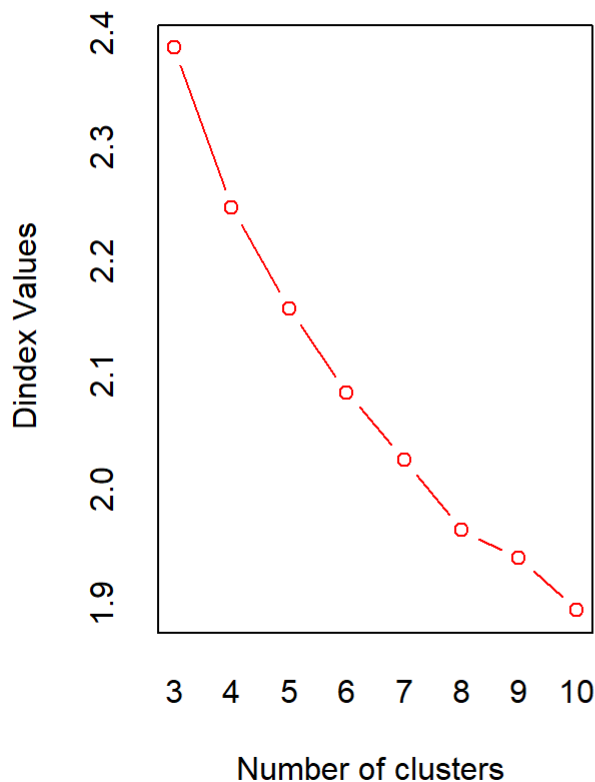
```
## Gap - max-gap k: 10 | 1-SE k: 1
```

Section 6: NbClust

```
set.seed(24)
nb <- NbClust(data = scaled, distance = "euclidean",
              min.nc = min(K_RANGE), max.nc = max(K_RANGE), method = "kmeans", index = "all")
```

```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##           In the plot of Hubert index, we seek a significant knee that corresponds t
o a
##           significant increase of the value of the measure i.e the significant peak
in Hubert
##           index second differences plot.
##
```

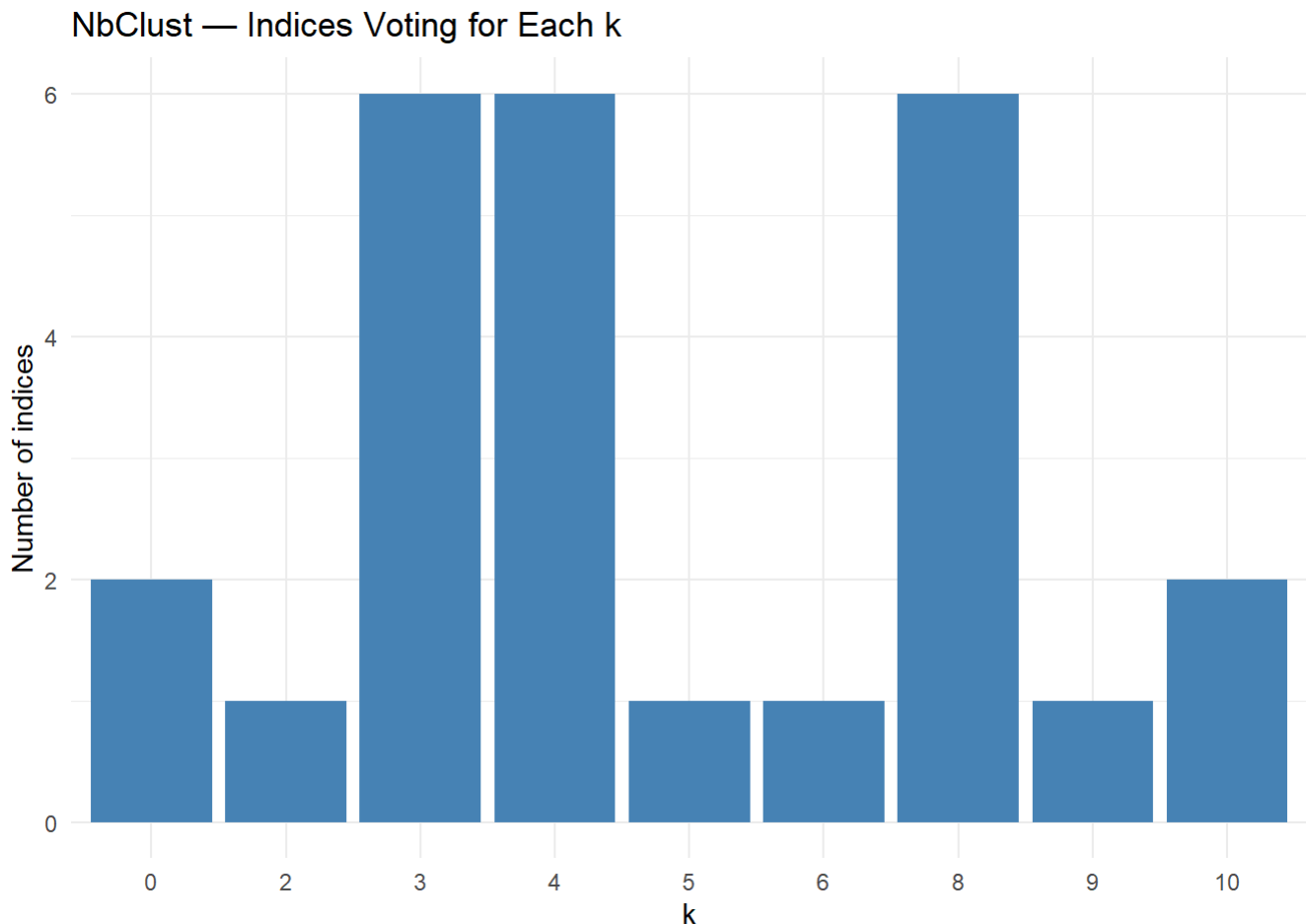


```
## *** : The D index is a graphical method of determining the number of clusters.
##           In the plot of D index, we seek a significant knee (the significant peak in
n Dindex
##           second differences plot) that corresponds to a significant increase of the
value of
##           the measure.
##
## *****
## * Among all indices:
## * 6 proposed 3 as the best number of clusters
## * 6 proposed 4 as the best number of clusters
## * 1 proposed 5 as the best number of clusters
## * 1 proposed 6 as the best number of clusters
## * 6 proposed 8 as the best number of clusters
## * 1 proposed 9 as the best number of clusters
## * 2 proposed 10 as the best number of clusters
##
##           ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is 3
##
## *****
```

```
nb_best_k <- as.numeric(names(which.max(table(nb$Best.nc[1, ]))))
cat("NbClust majority-vote best k:", nb_best_k, "\n")
```

```
## NbClust majority-vote best k: 3
```

```
nb_votes <- as.data.frame(table(k = nb$Best.nc[1, ]))
ggplot(nb_votes, aes(k, Freq)) +
  geom_col(fill = "steelblue") +
  labs(title = "NbClust – Indices Voting for Each k", x = "k", y = "Number of indices") +
  theme_minimal()
```



Section 7: Fuzzy C-Means

```
fcm_sil <- c(); fcm_models <- list()
for (k in K_RANGE) {
  set.seed(24 + k)
  fm <- cmeans(scaled, centers = k, iter.max = 1000, m = 2, method = "cmeans")
  fcm_models[[as.character(k)]] <- fm
  fcm_sil[as.character(k)] <- mean(silhouette(fm$cluster, dmat)[, 3])
}
fcm_table <- data.frame(k = K_RANGE, avg_silhouette = round(fcm_sil, 4))
print(fcm_table, row.names = FALSE)
```

```
## k avg_silhouette
## 3      0.0672
## 4      0.0255
## 5      0.0526
## 6     -0.1028
## 7     -0.0540
## 8     -0.0833
## 9      0.0583
## 10     0.0028
```

```
fcm_best_k <- fcm_table$k[which.max(fcm_table$avg_silhouette)]
fcm_best <- fcm_models[[as.character(fcm_best_k)]]
cat("Best k (Fuzzy C-Means):", fcm_best_k, "\n")
```

```
## Best k (Fuzzy C-Means): 3
```

```
set.seed(24)
gap_fcm <- clusGap(scaled, FUNcluster = function(x, k) {
  list(cluster = cmeans(x, centers = k, iter.max = 1000, m = 2, method = "cmeans")$cluster)
}, K.max = max(K_RANGE), B = GAP_B)
gap_fcm_res <- get_gap_best_k(gap_fcm)
cat("Gap - max-gap k:", gap_fcm_res$max_gap_k, "| 1-SE k:", gap_fcm_res$best_k_1se, "\n")
```

```
## Gap - max-gap k: 7 | 1-SE k: 1
```

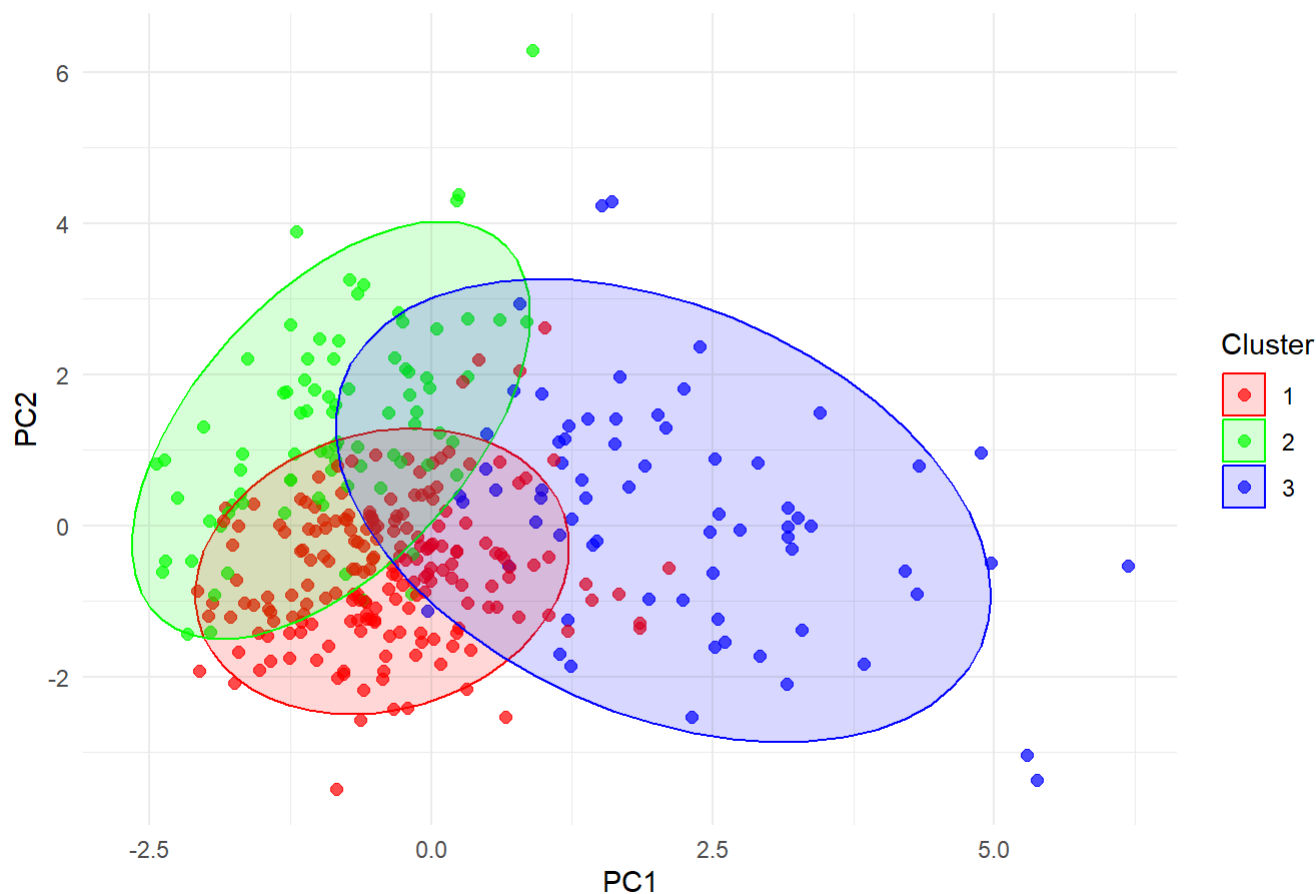
Section 8: Main Plots — PAM Manhattan

```
cluster_vec_man <- factor(pam_man_best$clustering)

# PCA
pca <- prcomp(scaled)
pca_df <- data.frame(PC1 = pca$x[,1], PC2 = pca$x[,2], Cluster = cluster_vec_man)
cluster_colors <- rainbow(nlevels(cluster_vec_man)); names(cluster_colors) <- levels(cluster_vec_man)

ggplot(pca_df, aes(PC1, PC2, color = Cluster, fill = Cluster)) +
  geom_point(alpha = 0.7, size = 2) +
  stat_ellipse(geom = "polygon", alpha = 0.15, level = 0.95) +
  scale_color_manual(values = cluster_colors) + scale_fill_manual(values = cluster_colors) +
  labs(title = paste("PCA Cluster Plot - PAM Manhattan (k =", pam_man_best_k, ")")) +
  theme_minimal() + theme(legend.position = "right")
```

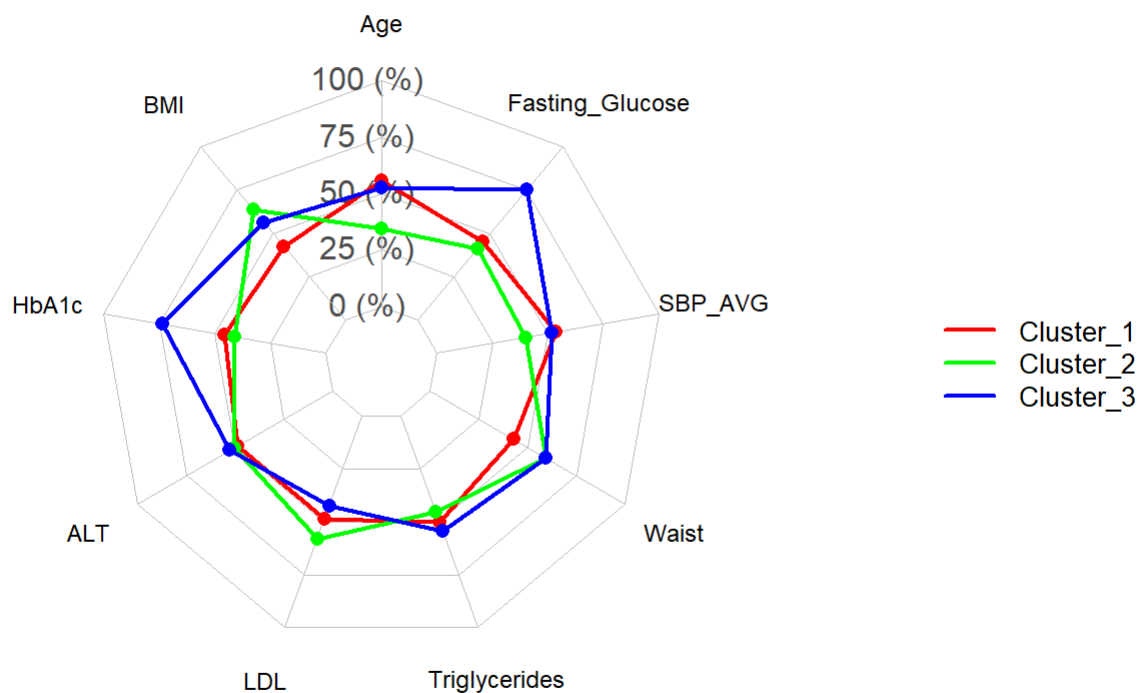
PCA Cluster Plot - PAM Manhattan (k = 3)



```
# Radar chart
z_profile <- aggregate(as.data.frame(scaled), by = list(Cluster = cluster_vec_man), FUN = mean)
radar_data <- z_profile[, clustering_vars]
rownames(radar_data) <- paste0("Cluster_", z_profile$Cluster)
radar_plot_df <- rbind(rep(3, ncol(radar_data)), rep(-3, ncol(radar_data)), radar_data)
radar_colors <- rainbow(nrow(radar_data))

old_par <- par(no.readonly = TRUE)
layout(matrix(c(1,2), nrow = 1), widths = c(3,1))
par(mar = c(2,2,4,1))
radarchart(radar_plot_df, axistype = 1, pcol = radar_colors, plwd = 2, plty = 1,
           cglcol = "grey", cglty = 1, axislabcol = "grey30", cglwd = 0.8, vlce = 0.75,
           title = "Radar Chart (Z-score) - PAM Manhattan")
par(mar = c(2,0,4,0)); plot.new()
legend("center", legend = rownames(radar_data), col = radar_colors, lty = 1, lwd = 2, bty =
      "n", cex = 0.85)
```

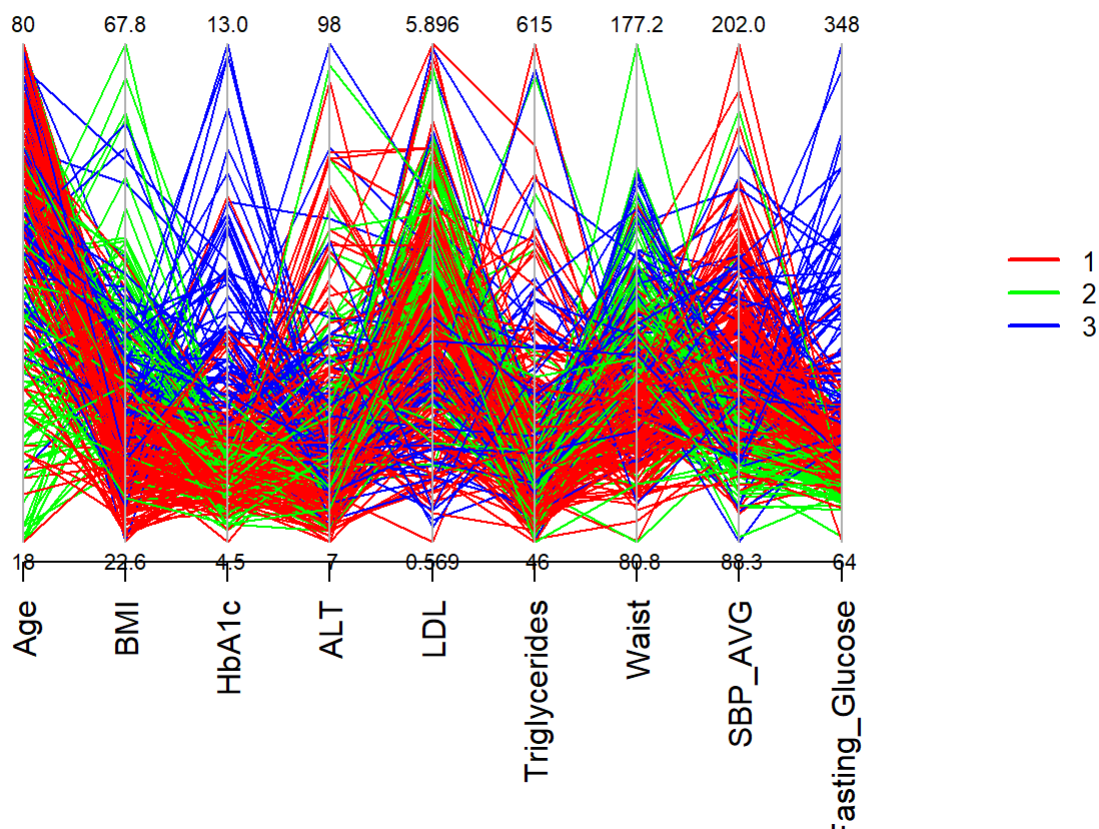
Radar Chart (Z-score) - PAM Manhattan



```
par(old_par); layout(1)

# Parallel coordinates
plot_colors_man <- rainbow(nlevels(cluster_vec_man))[as.integer(cluster_vec_man)]
old_par <- par(no.readonly = TRUE)
layout(matrix(c(1,2), nrow = 1), widths = c(4,1))
par(mar = c(7,4,4,1), las = 2)
MASS::parcoord(df[, clustering_vars], col = plot_colors_man, var.label = TRUE)
title(main = "Parallel Coordinate Plot - PAM Manhattan", line = 2)
par(mar = c(7,0,4,0), las = 1); plot.new()
legend("center", legend = levels(cluster_vec_man), col = rainbow(nlevels(cluster_vec_man)),
      lty = 1, lwd = 2, cex = 0.85, bty = "n")
```

Parallel Coordinate Plot - PAM Manhattan



```
par(old_par); layout(1)
```

```
# Cluster profile boxplots
```

```
raw_with_cluster <- as.data.frame(df[, clustering_vars]); raw_with_cluster$Cluster <- cluster_vec_man
```

```
raw_long <- melt(raw_with_cluster, id.vars = "Cluster", measure.vars = clustering_vars,  
                variable.name = "Variable", value.name = "Value")
```

```
ggplot(raw_long, aes(Cluster, Value, fill = Cluster)) +
```

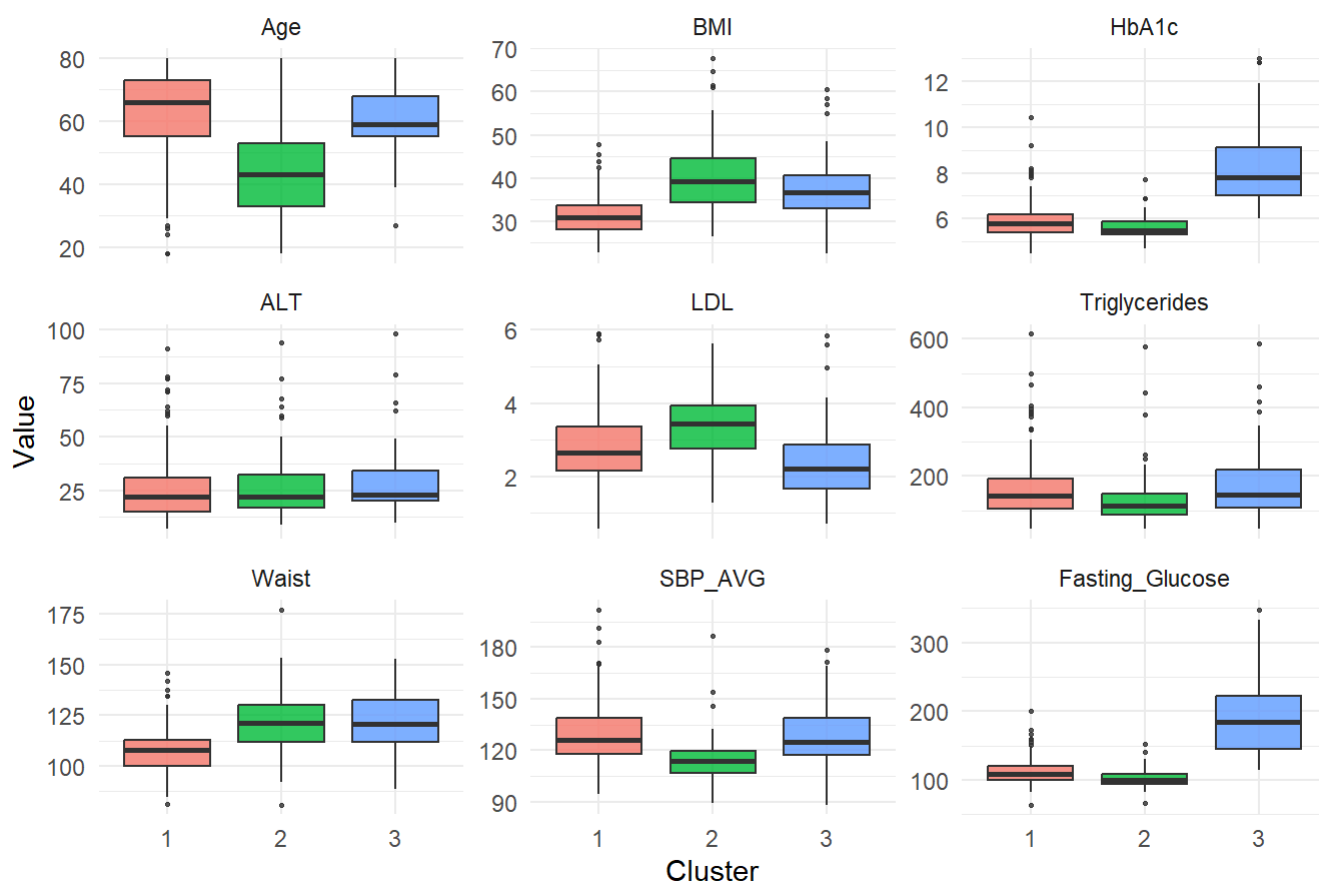
```
  geom_boxplot(outlier.size = 0.6, alpha = 0.8) +
```

```
  facet_wrap(~ Variable, scales = "free_y") +
```

```
  labs(title = "Cluster Profile Boxplots (raw units) - PAM Manhattan") +
```

```
  theme_minimal() + theme(legend.position = "none")
```

Cluster Profile Boxplots (raw units) - PAM Manhattan



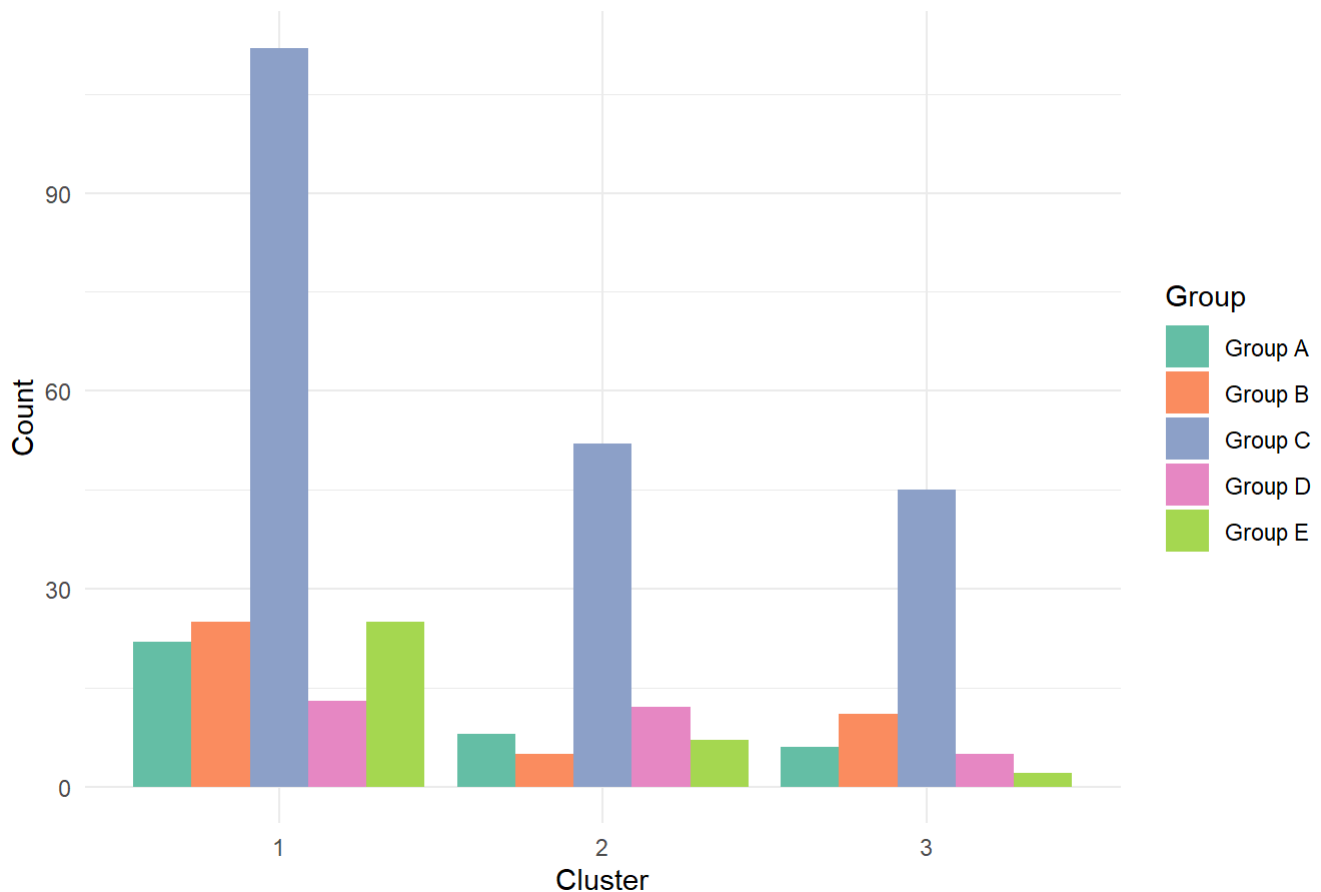
Section 8b: Population Group Distribution (Bar + Pie by Cluster)

```
group_labels <- c("1" = "Group A", "2" = "Group B", "3" = "Group C", "4" = "Group D", "5" = "Group E")
group_lookup <- data.frame(SEQN = df$SEQN, Cluster = pam_man_best$clustering)
group_merged <- merge(masld_clean[, c("SEQN", "Race")], group_lookup, by = "SEQN", all.y = TRUE)
group_merged$Group_Full <- group_labels[as.character(group_merged$Race)]

group_dist <- as.data.frame(table(Group = factor(group_merged$Group_Full, levels = group_labels),
                                           Cluster = factor(group_merged$Cluster)))

# Bar chart
ggplot(group_dist, aes(Cluster, Freq, fill = Group)) +
  geom_col(position = "dodge") + scale_fill_brewer(palette = "Set2") +
  labs(title = "Population Group Distribution by Cluster", x = "Cluster", y = "Count", fill = "Group") +
  theme_minimal() + theme(legend.position = "right")
```

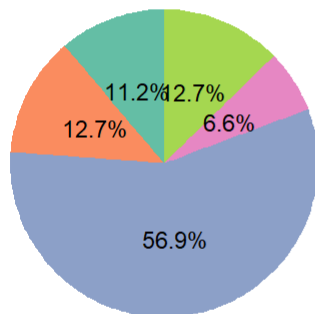

Population Group Distribution by Cluster



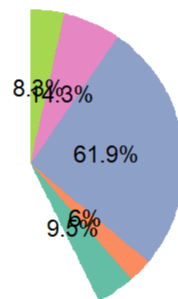
```
# Pie chart per cluster, faceted
group_dist_pct <- group_dist %>%
  group_by(Cluster) %>%
  mutate(Pct = round(100 * Freq / sum(Freq), 1)) %>%
  ungroup()

ggplot(group_dist_pct, aes(x = "", y = Freq, fill = Group)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar("y", start = 0) +
  facet_wrap(~ Cluster, labeller = label_both) +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Population Group Composition by Cluster (%)") +
  theme_void() +
  theme(legend.position = "right",
        strip.text = element_text(face = "bold", size = 11)) +
  geom_text(aes(label = ifelse(Pct >= 5, paste0(Pct, "%"), "")),
            position = position_stack(vjust = 0.5), size = 3)
```

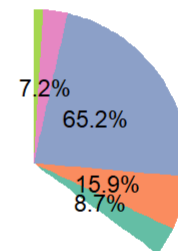
Population Group Composition by Cluster (%)



Cluster: 2



Cluster: 3



Group



```
cat("\nGroup distribution by cluster (raw counts):\n")
```

```
##
## Group distribution by cluster (raw counts):
```

```
print(table(group_merged$Group_Full, group_merged$Cluster, useNA = "ifany"))
```

```
##
##           1   2   3
## Group A   22   8   6
## Group B   25   5  11
## Group C  112  52  45
## Group D   13  12   5
## Group E   25   7   2
```

Section 9: Cross-Method Agreement + Stability (1000x 80% Subsample ARI)

```
ari <- function(a, b) round(adjustedRandIndex(a, b), 4)

cat("\n--- Cross-method agreement (single run) ---\n")
```

```
##
## --- Cross-method agreement (single run) ---
```

```
cat("PAM Manhattan vs PAM Euclidean:", ari(pam_man_best$clustering, pam_euc_best$clustering),
"\n")
```

```
## PAM Manhattan vs PAM Euclidean: 0.6966
```

```
cat("PAM Manhattan vs K-means:      ", ari(pam_man_best$clustering, km_best$cluster), "\n")
```

```
## PAM Manhattan vs K-means:      0.4566
```

```
cat("PAM Manhattan vs Fuzzy C-Means:", ari(pam_man_best$clustering, fcm_best$cluster), "\n")
```

```
## PAM Manhattan vs Fuzzy C-Means: 0.124
```

```
set.seed(24)
n_total <- nrow(scaled)
subsample_size <- floor(0.8 * n_total)
n_iter <- 1000
stability_ari <- numeric(n_iter)
reference_clusters <- pam_man_best$clustering
names(reference_clusters) <- rownames(scaled)

for (i in 1:n_iter) {
  idx <- sample(1:n_total, subsample_size, replace = FALSE)
  sub_data <- scaled[idx, ]
  sub_model <- tryCatch(pam(sub_data, k = pam_man_best_k, metric = "manhattan", nstart = 5),
    error = function(e) NULL)
  stability_ari[i] <- if (!is.null(sub_model)) {
    ari(reference_clusters[rownames(sub_data)], sub_model$clustering)
  } else NA
}

cat("\n=== STABILITY (1000x 80% subsample, PAM Manhattan k =", pam_man_best_k, ") ===\n")
```

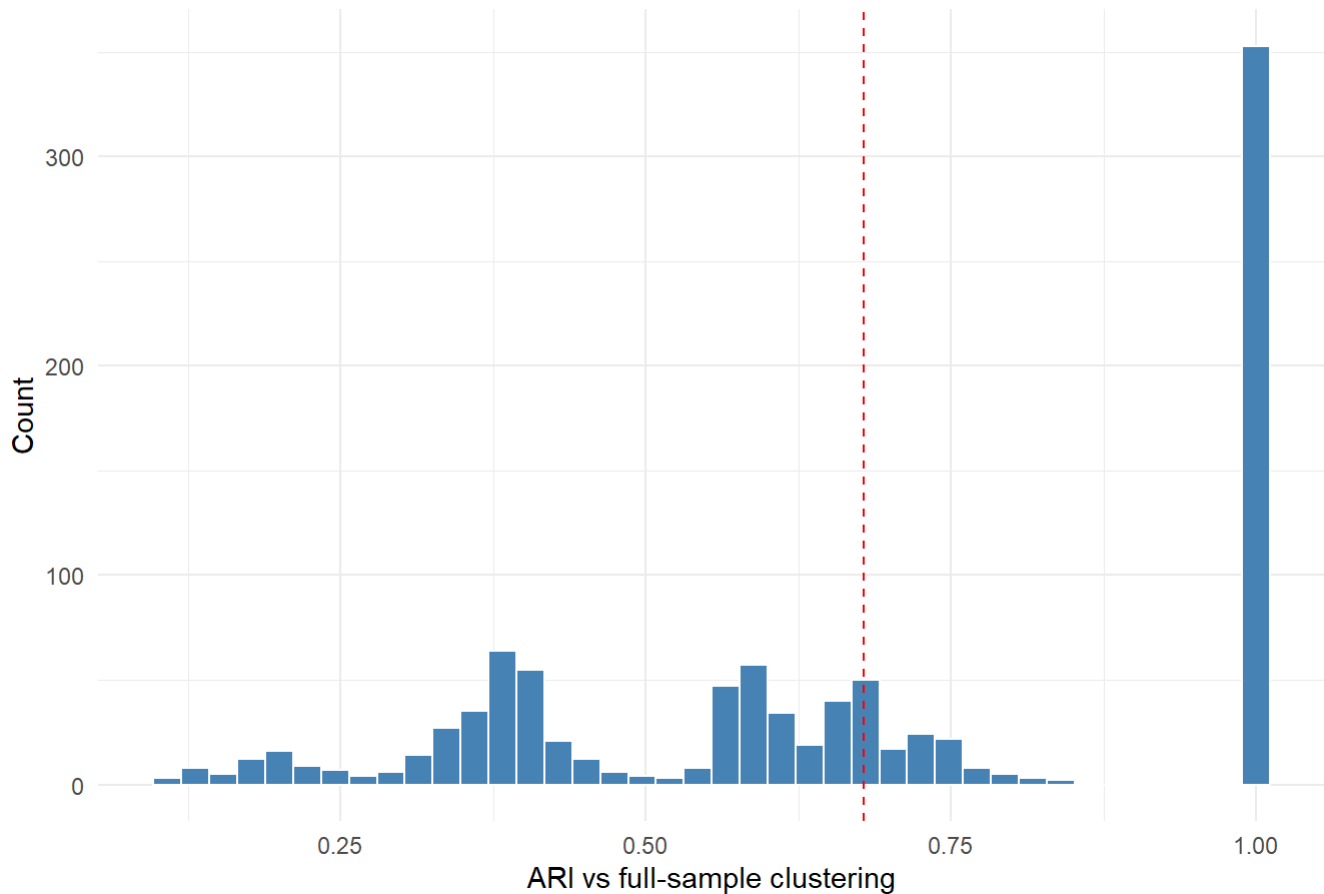
```
##
## === STABILITY (1000x 80% subsample, PAM Manhattan k = 3 ) ===
```

```
cat("Mean ARI:", round(mean(stability_ari, na.rm = TRUE), 4),
    "| Median:", round(median(stability_ari, na.rm = TRUE), 4),
    "| SD:", round(sd(stability_ari, na.rm = TRUE), 4), "\n")
```

```
## Mean ARI: 0.6786 | Median: 0.6596 | SD: 0.2739
```

```
ggplot(data.frame(ARI = stability_ari), aes(x = ARI)) +
  geom_histogram(bins = 40, fill = "steelblue", color = "white") +
  geom_vline(xintercept = mean(stability_ari, na.rm = TRUE), color = "red", linetype = "dashed") +
  labs(title = paste("Cluster Stability – 1000x 80% Subsample ARI (k =", pam_man_best_k,
    ")"),
    x = "ARI vs full-sample clustering", y = "Count") +
  theme_minimal()
```

Cluster Stability — 1000x 80% Subsample ARI (k = 3)



Section 10: Cluster Characterization (auto-derived

from results)

```
# For each cluster, list which clustering variables are most elevated (>+0.3 SD)
# and most reduced (<-0.3 SD) relative to the overall sample mean – this turns
# the z-score profile into a plain-language summary automatically, rather than
# requiring a manual read of the radar chart each time.
```

```
z_profile_named <- z_profile
rownames(z_profile_named) <- paste("Cluster", z_profile_named$Cluster)
```

```
cluster_summary_table <- df
cluster_summary_table$Cluster <- cluster_vec_man
```

```
descriptive_table <- cluster_summary_table %>%
  group_by(Cluster) %>%
  summarise(across(all_of(clustering_vars),
    list(median = ~median(.x, na.rm = TRUE),
         IQR = ~IQR(.x, na.rm = TRUE)))) %>%
  as.data.frame()
```

```
cat("=== DESCRIPTIVE SUMMARY (median [IQR]) BY CLUSTER ===\n")
```

```
## === DESCRIPTIVE SUMMARY (median [IQR]) BY CLUSTER ===
```

```
print(descriptive_table, row.names = FALSE)
```

```
## Cluster Age_median Age_IQR BMI_median BMI_IQR HbA1c_median HbA1c_IQR
##      1      66      18      30.9  5.500      5.8      0.8
##      2      43      20      39.2 10.125      5.5      0.6
##      3      59      13      36.6  7.700      7.8      2.1
## ALT_median ALT_IQR LDL_median LDL_IQR Triglycerides_median Triglycerides_IQR
##      22  16.00    2.664  1.190      144      87.00
##      22  15.25    3.439  1.164      115      62.25
##      23  14.00    2.224  1.215      147     112.00
## Waist_median Waist_IQR SBP_AVG_median SBP_AVG_IQR Fasting_Glucose_median
##      107.6  12.500      126.3333  21.00000      108
##      121.3  18.275      114.0000  12.75000      100
##      120.5  20.950      124.6667  21.33333      185
## Fasting_Glucose_IQR
##      20.00
##      15.25
##      78.00
```

```
cat("\n=== AUTO-GENERATED CLUSTER PROFILE SUMMARY ===\n")
```

```
##
## === AUTO-GENERATED CLUSTER PROFILE SUMMARY ===
```

```

for (i in seq_len(nrow(z_profile))) {
  cl <- z_profile$Cluster[i]
  vals <- unlist(z_profile[i, clustering_vars])
  elevated <- names(sort(vals[vals > 0.3], decreasing = TRUE))
  reduced <- names(sort(vals[vals < -0.3]))
  n_members <- sum(cluster_vec_man == cl)

  cat("\n--- Cluster", cl, "(n =", n_members, ") ---\n")
  if (length(elevated) > 0) {
    cat("  Elevated (>+0.3 SD):", paste(elevated, collapse = ", "), "\n")
  } else {
    cat("  Elevated (>+0.3 SD): none\n")
  }
  if (length(reduced) > 0) {
    cat("  Reduced (<-0.3 SD):", paste(reduced, collapse = ", "), "\n")
  } else {
    cat("  Reduced (<-0.3 SD): none\n")
  }
}

```

```

##
## --- Cluster 1 (n = 197 ) ---
##   Elevated (>+0.3 SD): Age
##   Reduced (<-0.3 SD): BMI, Waist, Fasting_Glucose
##
## --- Cluster 2 (n = 84 ) ---
##   Elevated (>+0.3 SD): BMI, Waist, LDL
##   Reduced (<-0.3 SD): Age, SBP_AVG, HbA1c, Fasting_Glucose
##
## --- Cluster 3 (n = 69 ) ---
##   Elevated (>+0.3 SD): Fasting_Glucose, HbA1c, Waist, BMI
##   Reduced (<-0.3 SD): LDL

```

Final Summary

```

cat("\n===== \nSUMMARY \n=====
=====\n")

```

```

##
## =====
## SUMMARY
## =====

```

```

cat("PAM Manhattan (MAIN) – best k:", pam_man_best_k, "| silhouette:", round(max(pam_man_si
1), 4),
    "| Gap 1-SE k:", gap_pam_man_res$best_k_1se, "\n")

```

```

## PAM Manhattan (MAIN) – best k: 3 | silhouette: 0.1544 | Gap 1-SE k: 1

```

```
cat("PAM Euclidean          – best k:", pam_euc_best_k, "| Gap 1-SE k:", gap_pam_euc_res$best_k_1se, "\n")
```

```
## PAM Euclidean          – best k: 3 | Gap 1-SE k: 1
```

```
cat("K-means                – best k:", km_best_k, "| Gap 1-SE k:", gap_km_res$best_k_1se, "\n")
```

```
## K-means                – best k: 3 | Gap 1-SE k: 1
```

```
cat("NbClust majority vote – best k:", nb_best_k, "\n")
```

```
## NbClust majority vote – best k: 3
```

```
cat("Fuzzy C-Means          – best k:", fcm_best_k, "| Gap 1-SE k:", gap_fcm_res$best_k_1se, "\n")
```

```
## Fuzzy C-Means          – best k: 3 | Gap 1-SE k: 1
```

```
cat("Stability (1000x 80% ARI, mean):", round(mean(stability_ari, na.rm = TRUE), 4), "\n")
```

```
## Stability (1000x 80% ARI, mean): 0.6786
```